Remediation Plan

Web Conference Application

Addressing findings from the Standards Compliance Report
of March 2, 2026

Application	web-conference
Plan Date	March 3, 2026
Compliance Report	`compliance-reports/compliance-report.tex`
Findings Addressed	58 (25 non-compliant, 33 partial)
Recommendations	21 consolidated items
Implementation Phases	4 (Critical, High, Medium, Low)

Phase	Items	Est. Effort	Findings Resolved
Phase 1 — Critical	3	Small–Medium	18
Phase 2 — High Priority	5	Small–Medium	22
Phase 3 — Medium Priority	3	Trivial–Medium	8
Phase 4 — Low Priority	7	Trivial–Medium	10
Total	**18**		**58**

Companion document to the Standards Compliance Report.
Addresses all 20 consolidated recommendations across 8 applicable standards.

Contents

1 Executive Summary **3**
- 1.1 Motivation . . . 3
- 1.2 Audio/Video Failure Analysis . . . 3
- 1.3 Plan Structure . . . 4

2 Phase 1: Critical — Connectivity and Core Security **6**
- 2.1 1.1 Add TURN Server Configuration . . . 6
  - 2.1.1 Problem . . . 6
  - 2.1.2 Proposed Changes . . . 6
  - 2.1.3 TURN Server Options . . . 7
- 2.2 1.2 Restrict CORS to Trusted Origins . . . 8
  - 2.2.1 Problem . . . 8
  - 2.2.2 Proposed Changes . . . 8
- 2.3 1.3 Implement Room Access Control . . . 10
  - 2.3.1 Problem . . . 10
  - 2.3.2 Proposed Changes . . . 10
  - 2.3.3 Scope Limitation . . . 11

3 Phase 2: High Priority — Connection Resilience **12**
- 3.1 2.1 Implement ICE Restart . . . 12
  - 3.1.1 Problem . . . 12
  - 3.1.2 Proposed Changes . . . 12
- 3.2 2.2 Differentiate `disconnected` from `failed` . . . 14
  - 3.2.1 Problem . . . 14
  - 3.2.2 Proposed Changes . . . 14
- 3.3 2.3 Implement `onnegotiationneeded` with Glare Handling . . . 16
  - 3.3.1 Problem . . . 16
  - 3.3.2 Proposed Changes . . . 16
- 3.4 2.4 Enforce HTTPS at Application Level . . . 18
  - 3.4.1 Problem . . . 18
  - 3.4.2 Proposed Changes . . . 18
- 3.5 2.5 Add IP Address Privacy Controls . . . 18
  - 3.5.1 Problem . . . 19
  - 3.5.2 Proposed Changes . . . 19

4 Phase 3: Medium Priority — Code Quality and Standards Compliance **20**
- 4.1 3.1 Remove Deprecated Constructor Usage . . . 20
  - 4.1.1 Problem . . . 20
  - 4.1.2 Proposed Changes . . . 20
- 4.2 3.2 Add Encryption Status UI . . . 21
  - 4.2.1 Problem . . . 21
  - 4.2.2 Proposed Changes . . . 21
- 4.3 3.3 Optimize Muted Track Bandwidth . . . 21
  - 4.3.1 Problem . . . 21
  - 4.3.2 Proposed Changes . . . 22

5 Phase 4: Low Priority — Polish and Monitoring **23**
5.1 4.1 Add `getStats()` Connection Quality Monitoring . . . 23
5.1.1 Proposed Changes . . . 23
5.2 4.2 Enhance Error Handling with User Notifications . . . 23
5.2.1 Proposed Changes . . . 23
5.3 4.3 Add `beforeunload` Cleanup Handler . . . 24
5.3.1 Proposed Changes . . . 24
5.4 4.4 Make ICE Server Configuration Dynamic . . . 24
5.4.1 Note . . . 24
5.5 4.5 Add `replaceTrack` Error Handling . . . 24
5.5.1 Proposed Changes . . . 25
5.6 4.6 Enhance Screen Sharing Warnings . . . 25
5.6.1 Proposed Changes . . . 25
5.7 4.7 Connection Analytics Panel . . . 25
5.7.1 Problem . . . 25
5.7.2 Proposed Changes . . . 25
5.7.3 UI Design . . . 27

6 Implementation Summary **28**
6.1 Files Modified Per Phase . . . 28
6.2 New Dependencies . . . 28
6.3 New Environment Variables . . . 29

7 Verification Plan **30**

8 Standards Coverage Matrix **32**

Executive Summary

This document presents a phased remediation plan for the **web-conference** application, addressing all 58 non-compliant and partially compliant findings identified in the Standards Compliance Report of March 2, 2026.

Motivation

The compliance report assessed the application against 8 applicable standards (5 direct, 3 indirect) and identified critical gaps in three areas:

1. **Connectivity** — The absence of TURN servers causes complete media failure for users behind symmetric NATs or restrictive firewalls. Users can join the signaling channel (Socket.IO over HTTP) but receive zero audio and video because the WebRTC peer-to-peer media path cannot be established. This is the most likely root cause of reported "join but no audio/video" incidents.
2. **Security** — Wildcard CORS (`origin: '*'`), no user authentication, and HTTP-only signaling create an open attack surface exploitable by any website.
3. **Resilience** — Aggressive peer removal on transient connection state changes, no ICE restart capability, and no renegotiation support mean the application cannot recover from temporary network disruptions.

Audio/Video Failure Analysis

Users have reported being able to join a conference room but experiencing no audio or video. The following findings from the compliance report explain this behavior:

Priority	Root Cause	Mechanism
CRITIC	No TURN servers	Users behind symmetric NATs or firewalls connect to the signaling server (Socket.IO over HTTP works through any firewall) but the WebRTC peer connection fails because ICE can only gather host and server-reflexive candidates via STUN. Without TURN relay candidates, there is no fallback path. The media connection silently fails: no audio, no video. *This is the most probable cause.*
HIGH	No ICE restart	When a connection momentarily degrades, the peer is permanently removed instead of attempting recovery via `restartIce()` or `createOffer({ iceRestart: true })`.
HIGH	`disconnected` = `failed`	The `disconnected` state (often transient) is treated identically to `failed` (permanent), causing premature peer removal during brief network fluctuations.
MEDIU	No ICE state monitoring	No `oniceconnectionstatechange` handler means no visibility into why ICE failed or how to recover.
MEDIU	No `getStats()`	No connection quality monitoring means no way to detect or diagnose degraded connections before they fail.
LOW	Browser permissions	If a user never responds to the browser's camera/microphone permission prompt, `getUserMedia()` hangs indefinitely and `joinRoom()` never executes. The user sees the room page but never actually joins. This is a browser UX issue, not a bug, but could be mitigated with a timeout and user guidance.

Key insight: The signaling channel (Socket.IO) and the media channel (WebRTC) use completely different network paths. Socket.IO connects over HTTP/WebSocket, which works through virtually any firewall. WebRTC requires a direct or relayed UDP/TCP path between peers, which fails without TURN in restrictive network environments. This is why users appear to "join" (signaling succeeds) but have no audio or video (media path fails).

Plan Structure

The 20 consolidated recommendations from the compliance report are organized into four implementation phases:

- **Phase 1 (Critical)** — Three items that fix connectivity failures and close critical security gaps. Phase 1 alone would resolve the reported audio/video issue for the majority of affected users.

- **Phase 2 (High Priority)** — Five items that improve connection resilience, enforce HTTPS, and add privacy controls.
- **Phase 3 (Medium Priority)** — Three items that improve code quality and standards compliance.
- **Phase 4 (Low Priority)** — Six items that add monitoring, polish error handling, and implement minor enhancements.

Phase 1: Critical — Connectivity and Core Security

These three items address the most impactful findings. Phase 1 directly fixes the "join but no audio/video" problem and closes the most dangerous security gaps.

1.1 Add TURN Server Configuration

Priority:	CRITICAL
Effort:	Small
Files:	`public/js/room.js`, `server/index.js`, `.env.example`
Findings:	F-002, F-029 (WebRTC 1.0); F-001 (WebRTC ED); F-001, F-002, F-015 (RFC 8445); F-006 (RFC 8826); F-008 (RFC 8827)
Standards:	6 of 8 applicable standards cite this issue

Problem

The application configures only Google STUN servers:

```
iceServers: [
    { urls: 'stun:stun.l.google.com:19302' },
    { urls: 'stun:stun1.l.google.com:19302' }
]
```

STUN servers only help discover a peer's public IP address. They cannot relay media traffic. When either peer is behind a symmetric NAT or restrictive firewall (common in corporate networks, many ISPs, and mobile carriers), STUN-only candidates cannot establish a direct path. Without TURN relay candidates, the WebRTC connection fails silently — no audio, no video.

Proposed Changes

1. **Server: Add `/api/ice-servers` endpoint** (`server/index.js`)

   Create a new API endpoint that returns the ICE server configuration dynamically. This ensures TURN credentials are never hardcoded in client JavaScript and enables server-side credential rotation.

   ```
   app.get('/api/ice-servers', (req, res) => {
       const iceServers = [
           { urls: 'stun:stun.l.google.com:19302' },
           { urls: 'stun:stun1.l.google.com:19302' }
       ];

       if (process.env.TURN_URL) {
           iceServers.push({
               urls: process.env.TURN_URL,
               username: process.env.TURN_USERNAME,
               credential: process.env.TURN_CREDENTIAL
           });
       }
       if (process.env.TURNS_URL) {
   ```

```
        iceServers.push({
            urls: process.env.TURNS_URL,
            username: process.env.TURN_USERNAME,
            credential: process.env.TURN_CREDENTIAL
        });
    }

    res.json({ iceServers });
});
```

2. **Client: Fetch ICE servers before joining** (`public/js/room.js`)

   Replace the hardcoded `iceServers` configuration with a dynamic fetch in the `init()` method. The `iceServers` property should be populated from the server response before any `RTCPeerConnection` is created.

   ```
   async fetchIceServers() {
       try {
           const response = await fetch('/api/ice-servers');
           const data = await response.json();
           this.iceServers = { iceServers: data.iceServers };
       } catch (error) {
           console.error('Failed to fetch ICE servers:', error);
           // Fall back to STUN-only
       }
   }
   ```

   Call `await this.fetchIceServers()` in `init()` before `getLocalMedia()`.

3. **Environment: Add TURN variables** (`.env.example`)

   ```
   TURN_URL=turn:your-turn-server.example.com:3478
   TURNS_URL=turns:your-turn-server.example.com:5349
   TURN_USERNAME=your-username
   TURN_CREDENTIAL=your-credential
   ```

TURN Server Options

Provider	Type	Notes
coturn	Self-hosted (open source)	Full control; requires server infrastructure
Metered TURN	Managed service	Pay-per-use; free tier available
Twilio Network Traversal	Managed service	Temporary credentials via REST API
Xirsys	Managed service	Global PoPs; straightforward API

1.2 Restrict CORS to Trusted Origins

Priority:	CRITICAL
Effort:	Trivial
Files:	`server/index.js`
Findings:	F-009, F-014 (RFC 8826); F-014 (RFC 8827); F-030 (WebRTC 1.0); F-035 (RFC 8829)
Standards:	Cited in 4 reports

Problem

Both the Express CORS middleware and the Socket.IO server are configured with `origin: '*'`:

```
const io = new Server(server, {
    cors: {
        origin: '*',
        methods: ['GET', 'POST']
    }
});
```

This allows any website on the internet to connect to the signaling server, send WebRTC offers, join rooms, and potentially intercept SDP containing DTLS fingerprints. Combined with no authentication, this creates an open signaling server exploitable by any origin.

Proposed Changes

1. **Add `ALLOWED_ORIGINS` environment variable** (`.env.example`)

   ```
   ALLOWED_ORIGINS=https://your-app.example.com
   ```

2. **Configure CORS dynamically** (`server/index.js`)

   ```
   const allowedOrigins = process.env.ALLOWED_ORIGINS
       ? process.env.ALLOWED_ORIGINS.split(',')
       : (process.env.NODE_ENV === 'production' ? [] : '*');

   const corsConfig = {
       origin: allowedOrigins === '*' ? '*' : (origin, callback) => {
           if (!origin || allowedOrigins.includes(origin)) {
               callback(null, true);
           } else {
               callback(new Error('CORS not allowed'));
           }
       },
       methods: ['GET', 'POST']
   };
   ```

   Apply `corsConfig` to both the Express middleware and the Socket.IO `Server` constructor.

3. **Default behavior:**
   - Development (no `ALLOWED_ORIGINS` set, `NODE_ENV` not `production`): allow all origins (preserves current development workflow)
   - Production (no `ALLOWED_ORIGINS` set, `NODE_ENV=production`): reject cross-origin requests (fail-safe)
   - Explicit configuration: honor the comma-separated list

1.3 Implement Room Access Control

Priority:	CRITICAL
Effort:	Medium
Files:	`server/index.js`, `public/index.html`, `public/js/main.js`, `public/js/room.js`
Findings:	F-010 (RFC 8826); F-015 (RFC 8827)
Standards:	Both security RFCs cite this as critical severity

Problem

There is no authentication or authorization. Anyone can join any room by knowing (or guessing) the room ID. Usernames are self-assigned strings stored in `sessionStorage` with no verification:

```
socket.on('join-room', ({ roomId, username }) => {
    socket.join(roomId);
    // No authentication check
    // ...
});
```

Proposed Changes

1. **Landing page: Add optional room password field** (`public/index.html`, `public/js/main.js`)

   Add a password input field to the join form. When creating a room, the password is optional. When joining an existing password-protected room, the password is required.

   Store the password in `sessionStorage` alongside the username so it can be passed to the signaling server when joining.

2. **Server: Validate room credentials on join** (`server/index.js`)

   Extend the room data structure to store a hashed password:

   ```
   const crypto = require('crypto');

   function hashPassword(password) {
       return crypto.createHash('sha256')
           .update(password).digest('hex');
   }

   socket.on('join-room', ({ roomId, username, password }) => {
       const room = rooms.get(roomId);

       if (room && room.password) {
           if (!password || hashPassword(password) !== room.password) {
               socket.emit('join-error', {
                   message: 'Incorrect room password'
               });
               return;
           }
       }
   ```

```
    // Create room with password if provided
    if (!room) {
        rooms.set(roomId, {
            participants: new Map(),
            password: password ? hashPassword(password) : null
        });
    }

    // ... proceed with join
});
```

3. **Client: Handle join errors** (`public/js/room.js`)

   Listen for `join-error` events and display the error to the user. If the password is wrong, redirect back to the landing page with an error message.

Scope Limitation

This implements room-level password protection, not full user authentication (OAuth, JWT sessions, etc.). Full user authentication is a larger architectural change documented as a future consideration but not included in this plan. Room passwords address the most critical finding ("anyone can join any room") with proportionate effort.

Phase 2: High Priority — Connection Resilience

These five items prevent the aggressive peer-removal behavior that makes transient network issues permanent, and enforce transport security.

2.1 Implement ICE Restart

Priority:	HIGH
Effort:	Small
Files:	`public/js/room.js`
Findings:	F-032 (WebRTC 1.0); F-015 (WebRTC ED); F-024, F-033 (RFC 8829); F-003, F-018 (RFC 8445)
Standards:	Cited in 4 reports

Problem

When a peer connection reaches the `failed` state, the peer is immediately and permanently removed:

```
connection.onconnectionstatechange = () => {
    if (connection.connectionState === 'disconnected' ||
        connection.connectionState === 'failed') {
        this.removePeer(userId);
    }
};
```

The WebRTC specification provides `restartIce()` and `createOffer({ iceRestart: true })` specifically for this scenario, but neither is used.

Proposed Changes

1. **Add ICE restart method with exponential backoff**

```
async attemptIceRestart(userId) {
    const peer = this.peers.get(userId);
    if (!peer) return;

    const maxRetries = 3;
    peer.iceRestartCount = (peer.iceRestartCount || 0) + 1;

    if (peer.iceRestartCount > maxRetries) {
        this.showToast(
            `Connection to ${peer.username} lost`, 'error');
        this.removePeer(userId);
        return;
    }

    const backoffMs = 1000 * Math.pow(2,
        peer.iceRestartCount - 1);
```

```
    this.showToast(
        `Reconnecting to ${peer.username}...`, 'info');

    await new Promise(r => setTimeout(r, backoffMs));

    try {
        const offer = await peer.connection.createOffer({
            iceRestart: true
        });
        await peer.connection.setLocalDescription(offer);
        this.socket.emit('offer', { to: userId, offer });
    } catch (error) {
        console.error('ICE restart failed:', error);
        this.removePeer(userId);
    }
}
```

2. **Reset restart counter on successful connection**

   When `connectionState` transitions to `connected`, reset `peer.iceRestartCount = 0`.

3. **Trigger ICE restart on `failed` state** (replaces immediate `removePeer`)

2.2 Differentiate `disconnected` from `failed`

Priority:	**HIGH**
Effort:	**Small**
Files:	`public/js/room.js`
Findings:	F-013, F-023 (WebRTC 1.0); F-018 (RFC 8445)

Problem

Both `disconnected` and `failed` states trigger immediate peer removal. Per the WebRTC specification, `disconnected` is often transient (e.g., brief WiFi dropout) and may self-resolve, while `failed` indicates ICE has exhausted all candidates.

Proposed Changes

Replace the current `onconnectionstatechange` handler with granular state handling:

```
connection.onconnectionstatechange = () => {
    const state = connection.connectionState;

    switch (state) {
        case 'connected':
            // Clear any pending disconnect timeout
            if (peer.disconnectTimeout) {
                clearTimeout(peer.disconnectTimeout);
                peer.disconnectTimeout = null;
            }
            // Reset ICE restart counter
            peer.iceRestartCount = 0;
            break;

        case 'disconnected':
            // Wait before taking action; may self-resolve
            peer.disconnectTimeout = setTimeout(() => {
                if (connection.connectionState === 'disconnected') {
                    this.attemptIceRestart(userId);
                }
            }, 10000); // 10-second grace period
            break;

        case 'failed':
            // Attempt ICE restart immediately
            this.attemptIceRestart(userId);
            break;

        case 'closed':
            this.removePeer(userId);
            break;
    }
```

```
};
```

Additionally, add an `oniceconnectionstatechange` handler for ICE-specific state monitoring:

```
connection.oniceconnectionstatechange = () => {
    console.log(`ICE state for ${username}: ` +
        connection.iceConnectionState);
};
```

2.3 Implement `onnegotiationneeded` with Glare Handling

Priority:	**HIGH**
Effort:	**Medium**
Files:	`public/js/room.js`
Findings:	F-014 (WebRTC 1.0); F-026 (WebRTC ED); F-023, F-028 (RFC 8829)
Standards:	Cited in 3 reports

Problem

No `onnegotiationneeded` handler exists. The application creates offers only at initial setup. This prevents mid-session renegotiation (e.g., adding/removing tracks) and leaves the application unable to handle "glare" scenarios where both peers send offers simultaneously.

Proposed Changes

Implement the **perfect negotiation pattern** as recommended by the W3C WebRTC specification:

1. **Track polite/impolite roles per peer**

   When creating a peer connection, store whether this peer is the "polite" side (the non-initiator, willing to roll back) or the "impolite" side (the initiator, ignores incoming offers during `have-local-offer`):

   ```
   const peer = {
       connection,
       username,
       polite: !initiator,
       makingOffer: false
   };
   ```

2. **Add `onnegotiationneeded` handler**

   ```
   connection.onnegotiationneeded = async () => {
       try {
           peer.makingOffer = true;
           await connection.setLocalDescription();
           this.socket.emit('offer', {
               to: userId,
               offer: connection.localDescription
           });
       } catch (error) {
           console.error('Negotiation error:', error);
       } finally {
           peer.makingOffer = false;
       }
   };
   ```

3. **Update offer handler with glare resolution**

```
this.socket.on('offer', async ({ from, username, offer }) => {
    const peer = this.peers.get(from) ||
        this.createPeerConnection(from, username, false);
    const conn = peer.connection;

    const offerCollision = peer.makingOffer ||
        conn.signalingState !== 'stable';

    const ignoreOffer = !peer.polite && offerCollision;
    if (ignoreOffer) return;

    try {
        await conn.setRemoteDescription(offer);
        this.flushPendingCandidates(from);
        if (conn.signalingState === 'have-remote-offer') {
            await conn.setLocalDescription();
            this.socket.emit('answer', {
                to: from,
                answer: conn.localDescription
            });
        }
    } catch (error) {
        console.error('Error handling offer:', error);
    }
});
```

4. **Remove explicit `createAndSendOffer` call for initiator**

   With `onnegotiationneeded` in place, the browser will fire the event automatically after `addTrack()`, making the explicit `createAndSendOffer()` call on the initiator redundant.

2.4 Enforce HTTPS at Application Level

Priority:	HIGH
Effort:	Small
Files:	`server/index.js`, `package.json`
Findings:	F-002, F-003 (RFC 8826); F-001 (RFC 8827); F-030 (WebRTC ED)

Problem

The server uses `http.createServer()` with no HTTPS enforcement. HTTPS termination is delegated entirely to Render.com. If accessed directly (or through a different hosting platform), all traffic — including SDP with DTLS fingerprints — transits in plaintext.

Proposed Changes

1. **Add `helmet` dependency** (`package.json`)

```
"helmet": "^7.1.0"
```

2. **Add HTTP-to-HTTPS redirect and security headers** (`server/index.js`)

```
const helmet = require('helmet');
app.use(helmet());

// Redirect HTTP to HTTPS in production
if (process.env.NODE_ENV === 'production') {
    app.use((req, res, next) => {
        if (req.headers['x-forwarded-proto'] !== 'https') {
            return res.redirect(301,
                'https://' + req.headers.host + req.url);
        }
        next();
    });
}
```

The `x-forwarded-proto` check works with reverse proxies (Render, Heroku, AWS ALB). Helmet adds HSTS, X-Frame-Options, X-Content-Type-Options, and other security headers.

2.5 Add IP Address Privacy Controls

Priority:	HIGH
Effort:	Small
Files:	`public/js/room.js`
Findings:	F-029 (WebRTC 1.0); F-006 (RFC 8826); F-008, F-018 (RFC 8827); F-010 (RFC 8445)
Prerequisite:	Requires TURN servers from item 1.1

Problem

The default `iceTransportPolicy: 'all'` exposes the user's real IP address via host ICE candidates. No option exists to restrict candidates to relay-only mode. ICE gathering begins immediately on room join with no user consent step.

Proposed Changes

1. **Add relay-only mode configuration**

   When the user opts in to enhanced privacy, set `iceTransportPolicy: 'relay'` in the `RTCConfiguration`. This ensures only TURN relay candidates are used, hiding the user's real IP address from peers.

   ```
   this.iceServers = {
       iceServers: data.iceServers,
       iceTransportPolicy: this.relayOnly ? 'relay' : 'all'
   };
   ```

2. **Expose as a UI toggle** (pre-join screen or room settings)

   Add a "Hide my IP address (relay only)" checkbox. This option requires TURN servers to be configured; if no TURN servers are available, the toggle should be disabled with a tooltip explaining why.

Phase 3: Medium Priority — Code Quality and Standards Compliance

3.1 Remove Deprecated Constructor Usage

Priority:	MEDIUM
Effort:	Trivial
Files:	`public/js/room.js`
Findings:	F-021 (WebRTC 1.0); F-009, F-012 (WebRTC ED); F-029 (RFC 8829)
Standards:	Cited in 3 reports

Problem

The code uses deprecated explicit constructors for `RTCSessionDescription` and `RTCIceCandidate`:

```
// Deprecated usage (current):
await peer.connection.setRemoteDescription(
    new RTCSessionDescription(offer));
await peer.connection.addIceCandidate(
    new RTCIceCandidate(candidate));
```

The WebRTC Editor's Draft deprecates these constructors. Modern browsers accept plain objects directly.

Proposed Changes

Replace all instances (5 locations in `room.js`) with plain objects:

```
// Correct usage:
await peer.connection.setRemoteDescription(offer);
await peer.connection.addIceCandidate(candidate);
```

Locations to update:

1. Line 363: `setRemoteDescription` in offer handler
2. Line 377: `setRemoteDescription` in answer handler
3. Line 390: `addIceCandidate` in ice-candidate handler
4. Line 477: `addIceCandidate` in `flushPendingCandidates`

3.2 Add Encryption Status UI

Priority: MEDIUM
Effort: Medium
Files: `public/js/room.js`, `public/css/style.css`
Findings: F-013 (RFC 8827); F-013 (RFC 3711)

Problem

No UI element indicates whether media streams are encrypted. Users have no way to verify that DTLS-SRTP is active. RFC 8827 recommends displaying security characteristics to the user.

Proposed Changes

1. **Add encryption verification via `getStats()`**

   Periodically call `getStats()` on each peer connection and check for the `transport` stat's `dtlsState === 'connected'` and `tlsVersion` fields.

2. **Display lock icon in video containers**

   Add a lock icon (`.encryption-status`) to each remote video container's `.video-status` area. Show a green lock when DTLS is confirmed active; show a warning icon otherwise.

3. **Add CSS for encryption indicator**

```
.encryption-status {
    font-size: 14px;
    opacity: 0.8;
}
.encryption-status.secure {
    color: var(--success-color);
}
.encryption-status.insecure {
    color: var(--danger-color);
}
```

3.3 Optimize Muted Track Bandwidth

Priority: MEDIUM
Effort: Small
Files: `public/js/room.js`
Findings: F-014 (RFC 3550)

Problem

Muting is implemented via `track.enabled = false`, which causes the browser to send black frames (video) or silence (audio) rather than stopping the RTP stream. This wastes bandwidth, especially for video.

Proposed Changes

When muting video, additionally deactivate the encoding via `RTCRtpSender.setParameters()`:

```
async toggleVideo() {
    if (this.localStream) {
        const videoTrack = this.localStream.getVideoTracks()[0];
        if (videoTrack) {
            this.isVideoEnabled = !this.isVideoEnabled;
            videoTrack.enabled = this.isVideoEnabled;

            // Deactivate encoding to save bandwidth
            this.peers.forEach((peer) => {
                const sender = peer.connection.getSenders()
                    .find(s => s.track?.kind === 'video');
                if (sender) {
                    const params = sender.getParameters();
                    params.encodings.forEach(e => {
                        e.active = this.isVideoEnabled;
                    });
                    sender.setParameters(params);
                }
            });

            this.updateVideoButton();
            this.socket.emit('toggle-video', {
                roomId: this.roomId,
                enabled: this.isVideoEnabled
            });
        }
    }
}
```

Phase 4: Low Priority — Polish and Monitoring

4.1 Add `getStats()` Connection Quality Monitoring

Priority: LOW
Effort: Medium
Files: `public/js/room.js`, `public/css/style.css`
Findings: F-028 (WebRTC 1.0)

Proposed Changes

1. Periodically call `getStats()` on each peer connection (every 5 seconds).
2. Extract key metrics: packet loss percentage, jitter, round-trip time (RTT), bytes sent/received.
3. Display a connection quality indicator in each video container:
   - Green bar: packet loss $< 1\%$, RTT < 150ms
   - Yellow bar: packet loss 1–5% or RTT 150–300ms
   - Red bar: packet loss $> 5\%$ or RTT > 300ms
4. Log metrics to console for debugging.

4.2 Enhance Error Handling with User Notifications

Priority: LOW
Effort: Small
Files: `public/js/room.js`
Findings: F-022 (WebRTC 1.0)

Proposed Changes

Replace `console.error`-only error handling with user-facing toast notifications for actionable errors:

Error Location	Current Handling	Proposed Handling
Offer creation	`console.error`	Toast: "Connection setup failed" + retry
Answer handling	`console.error`	Toast: "Peer connection error"
ICE candidate	`console.error`	Silent (non-critical)
Screen share	`console.error`	Toast: "Screen sharing failed" (unless user cancelled)
replaceTrack	No handling	Toast: "Failed to switch video source"

4.3 Add `beforeunload` Cleanup Handler

Priority: LOW
Effort: Trivial
Files: `public/js/room.js`
Findings: F-006 (WebRTC ED)

Proposed Changes

Add a `beforeunload` event listener in `setupEventListeners()` to gracefully close all peer connections when the tab or window is closed:

```
window.addEventListener('beforeunload', () => {
    this.peers.forEach((peer) => {
        peer.connection.close();
    });
    if (this.localStream) {
        this.localStream.getTracks().forEach(t => t.stop());
    }
    if (this.screenStream) {
        this.screenStream.getTracks().forEach(t => t.stop());
    }
});
```

This ensures peers receive a clean `closed` state transition rather than relying on the server's disconnect timeout to notify others.

4.4 Make ICE Server Configuration Dynamic

Priority: LOW
Effort: Small
Files: `public/js/room.js`, `server/index.js`
Findings: F-017 (RFC 8445)

Note

This item is largely addressed by item 1.1 (the `/api/ice-servers` endpoint). The remaining work is to support time-limited TURN credentials for managed TURN services (e.g., Twilio, which issues temporary credentials via REST API). The server endpoint would generate fresh credentials on each request rather than serving static values from environment variables.

4.5 Add `replaceTrack` Error Handling

Priority: LOW
Effort: Trivial
Files: `public/js/room.js`
Findings: F-013 (RFC 3550)

Proposed Changes

Wrap `replaceTrack()` calls in `startScreenShare()` and `stopScreenShare()` with try/catch and user notification:

```
try {
    await sender.replaceTrack(screenTrack);
} catch (error) {
    console.error('Failed to replace track:', error);
    this.showToast('Failed to switch video source', 'error');
}
```

Note: `replaceTrack()` returns a Promise, but the current code does not `await` it. The fix should also add `await`.

4.6 Enhance Screen Sharing Warnings

Priority:	LOW
Effort:	**Trivial**
Files:	`public/js/room.js`
Findings:	F-013 (RFC 8826)

Proposed Changes

Before calling `getDisplayMedia()`, show an application-level confirmation dialog warning the user about the risks of screen sharing (e.g., "Other participants will be able to see everything on your shared screen, including notifications and personal information.").

The browser already shows its own consent dialog for screen sharing, so this is an *additional* application-level warning. Implement using a simple `confirm()` dialog or a custom modal.

4.7 Connection Analytics Panel

Priority:	LOW
Effort:	**Medium**
Files:	`public/js/room.js`, `public/room.html`, `public/css/style.css`
Findings:	F-028 (WebRTC 1.0) — extends item 4.1

Problem

There is no visibility into connection internals from the application UI. Diagnosing connectivity issues (such as the "join but no audio/video" problem) currently requires navigating to `chrome://webrtc-internals` which is inaccessible to end users and unavailable in non-Chromium browsers.

Proposed Changes

Add a toggleable analytics panel in the room UI (activated via a control button or keyboard shortcut) that displays per-peer connection details sourced entirely from the browser's `RTCStatsReport`. No server changes required.

Network / ICE:

Metric	Stats API Source	Why It Matters
Local candidate type	`candidate-pair` → `local-candidate`	Shows if TURN relay is in use
Remote candidate type	`candidate-pair` → `remote-candidate`	Identifies peer's network path
TURN server address	Relay candidate `ip:port`	Confirms TURN connectivity
Selected candidate pair	`selectedCandidatePairId`	Active network path
Transport protocol	Candidate `protocol` (UDP/TCP)	Identifies transport type
ICE / connection state	`iceConnectionState`, `connectionState`	Current connection health

Quality Metrics (polled every 5 seconds):

Metric	Stats API Source	Notes
Round-trip time	`currentRoundTripTime`	On `candidate-pair`
Available bandwidth	`availableOutgoingBitrate`	On `candidate-pair`
Packet loss %	`packetsLost` / `packetsReceived`	On `inbound-rtp`
Jitter	`jitter`	On `inbound-rtp`
Bytes sent / received	`bytesSent`, `bytesReceived`	On `candidate-pair`

Media:

Metric	Stats API Source	Notes
Audio/video codec	`mimeType`	On `codec` stats
Video resolution (send/recv)	`frameWidth`, `frameHeight`	On `outbound-rtp` / `inbound-rtp`
Framerate	`framesPerSecond`	On `outbound-rtp` / `inbound-rtp`
Audio level	`audioLevel`	On `inbound-rtp` (audio)
Bitrate	Computed: Δbytes / Δtime	Per audio/video track

Security:

Metric	Stats API Source	Notes
DTLS state	`dtlsState`	On `transport`
DTLS cipher suite	`dtlsCipher`	On `transport`
SRTP cipher	`srtpCipher`	On `transport`
TLS version	`tlsVersion`	On `transport`

Connection:

- Time since connected (session duration)

- ICE candidates gathered by type (host / server-reflexive / relay counts)
- Candidate pairs checked vs. succeeded

UI Design

The panel should be togglable (hidden by default) via a control bar button. Display one section per remote peer, or an aggregated summary when multiple peers are connected. Use the existing dark theme styling. Auto-refresh metrics every 5 seconds while the panel is open; stop polling when closed to avoid unnecessary overhead.

Implementation Summary

Files Modified Per Phase

Phase	File	Changes
Phase	`server/index.js`	ICE servers endpoint, CORS restriction, room password validation, `join-error` event
Phase	`public/js/room.js`	Fetch ICE servers, pass password on join, handle join errors
Phase	`public/index.html`	Room password input field
Phase	`public/js/main.js`	Store/pass room password
Phase	`.env.example`	TURN and CORS environment variables
Phase	`public/js/room.js`	ICE restart, connection state handling, negotiationneeded, IP privacy toggle
Phase	`server/index.js`	Helmet middleware, HTTPS redirect
Phase	`package.json`	Add `helmet` dependency
Phase	`public/js/room.js`	Remove deprecated constructors, muted bandwidth optimization
Phase	`public/css/style.css`	Encryption status indicator styles
Phase	`public/js/room.js`	getStats monitoring, error handling, beforeunload, replaceTrack errors, screen share warnings, connection analytics panel
Phase	`public/room.html`	Analytics panel toggle button
Phase	`public/css/style.css`	Connection quality indicator and analytics panel styles

New Dependencies

Package	Version	Phase	Purpose
`helmet`	^7.1.0	Phase 2	Security headers (HSTS, X-Frame-Options, etc.)

New Environment Variables

Variable	Phase	Description
`TURN_URL`	1	TURN server URL (e.g., `turn:turn.example.com:3478`)
`TURNS_URL`	1	TURN-over-TLS URL (e.g., `turns:turn.example.com:5349`)
`TURN_USERNAME`	1	TURN server username
`TURN_CREDENTIAL`	1	TURN server credential
`ALLOWED_ORIGINS`	1	Comma-separated list of allowed CORS origins
`NODE_ENV`	2	Set to `production` to enable HTTPS redirect and strict CORS

Verification Plan

#	Test	Method	Expected Result
1	TURN connectivity	Two peers on different networks; one behind symmetric NAT or corporate firewall	Media flows via TURN relay; audio and video work
2	ICE restart recovery	Toggle WiFi off for 5 seconds, then back on	Connection recovers without peer removal; toast shows "Reconnecting..." then clears
3	Disconnect grace period	Simulate brief network interruption (< 10s)	Peer is not removed during `disconnected` state; recovers automatically
4	CORS rejection	Connect from unauthorized origin	Socket.IO connection rejected; HTTP requests return CORS error
5	Room password	Join password-protected room without credentials	`join-error` event received; user redirected to landing page with error message
6	Room password (correct)	Join with correct password	Normal join flow; audio and video work
7	HTTPS redirect	Access HTTP URL in production	Redirected to HTTPS with 301
8	Security headers	Inspect response headers in production	HSTS, X-Frame-Options, X-Content-Type-Options present
9	Deprecated constructors	Search codebase	No `new RTCSessionDescription` or `new RTCIceCandidate` calls remain
10	Renegotiation	Add/remove tracks mid-call	`onnegotiationneeded` fires; new offer/answer exchange succeeds
11	Glare handling	Both peers trigger renegotiation simultaneously	One side rolls back; connection remains stable
12	Relay-only mode	Enable "Hide my IP" toggle	Only relay candidates gathered (verify via `chrome://webrtc-internals`)

#	Test	Method	Expected Result
13	Encryption indicator	Establish connection	Lock icon appears in video container after DTLS handshake completes
14	Muted bandwidth	Mute video; check `getStats()`	Outbound video bitrate drops to near-zero (encoding deactivated)
15	Tab close cleanup	Close browser tab during active call	Other participants see peer removed promptly; no zombie connections
16	Connection quality	Simulate packet loss (e.g., `tc netem`)	Quality indicator changes from green to yellow/red
17	Connection analytics panel	Open analytics panel during active call	Panel shows candidate types, TURN address (if relay), RTT, packet loss, codec, encryption status

Standards Coverage Matrix

The following matrix maps each remediation item to the findings it resolves, grouped by standard. Items that resolve findings across multiple standards are highlighted.

#	Item	WebRTC	WebRTC ED	RFC 8829	RFC 8826	RFC 8827	RFC 8445	RFC 3550	RFC 3711
1.1	TURN servers	•	•	•	•	•	•		
1.2	Restrict CORS	•		•	•	•			
1.3	Room access				•	•			
2.1	ICE restart	•	•	•			•		
2.2	State handling	•					•		
2.3	Negotiation	•	•	•					
2.4	HTTPS		•		•	•			
2.5	IP privacy	•			•	•	•		
3.1	Constructors	•	•	•					
3.2	Encryption UI					•			•
3.3	Muted BW							•	
4.1	getStats	•						•	
4.2	Error handling	•							
4.3	beforeunload		•						
4.4	Dynamic ICE						•		
4.5	replaceTrack							•	
4.6	Screen warn				•				
4.7	Analytics panel	•						•	

— End of Remediation Plan —